



ԵՐԵՎԱՆԻ
ՊԵՏԱԿԱՆ
ՀԱՄԱԼՍԱՐԱՆ

ՀԵՏԱԶՈՏԱԿԱՆ ԱՇԽԱՏԱՆՔ

Թեմա՝ GRU-Gated Recurrent Unit

Ֆակուլտետ՝ Ինֆորմատիկա և կիրառական մաթեմատիկա համակարգչային գիտություն

Կուրս՝ 3-րդ

Առարկա՝ Մեքենայական ուսուցում

Ուսանողներ՝ Խաչանուշ Բլեյան, Էմմա Սադյան, Սվետլանա Հայրապետյան

Դասախոս՝ Ա. Հովհակիմյան

Երևան 2025

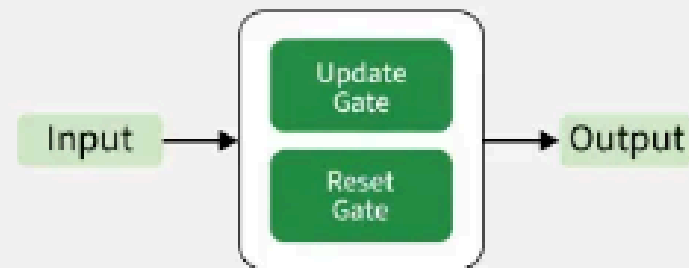
Ի՞նչ է GRU-ն

What are GRUs?

A type of RNN for sequential data

Uses 2 gates: update and reset

Faster and simpler than LSTM



Gated Recurrent Unit (GRU) – Ռեկուրենտ նեյրոնային ցանցերի (RNN) կատարելագործված տեսակ է, որը ստեղծվել է հաջորդական տվյալների արդյունավետ մշակման համար 2014 թվականին Cho et al. կողմից :

1. Պարզեցված կառուցվածք

GRU-ն ունի ավելի պարզ կառուցվածք քան LSTM-ը՝ օգտագործելով ընդամենը երկու դարպաս (Update Gate և Reset Gate) տեղեկատվության հոսքը կառավարելու համար, թե որքան տեղեկատվություն պետք է անցնի հաջորդ քայլին և որքանը պետք է ջնջվի :

2. Vanishing Gradient խնդրի լուծում

Ինչպես LSTM-ը, GRU-ն նույնպես լուծում է «անհետացող գրադիենտի» խնդիրը, ինչը թույլ է տալիս մոդելին հիշել երկարաժամկետ կախվածությունները տվյալների մեջ:

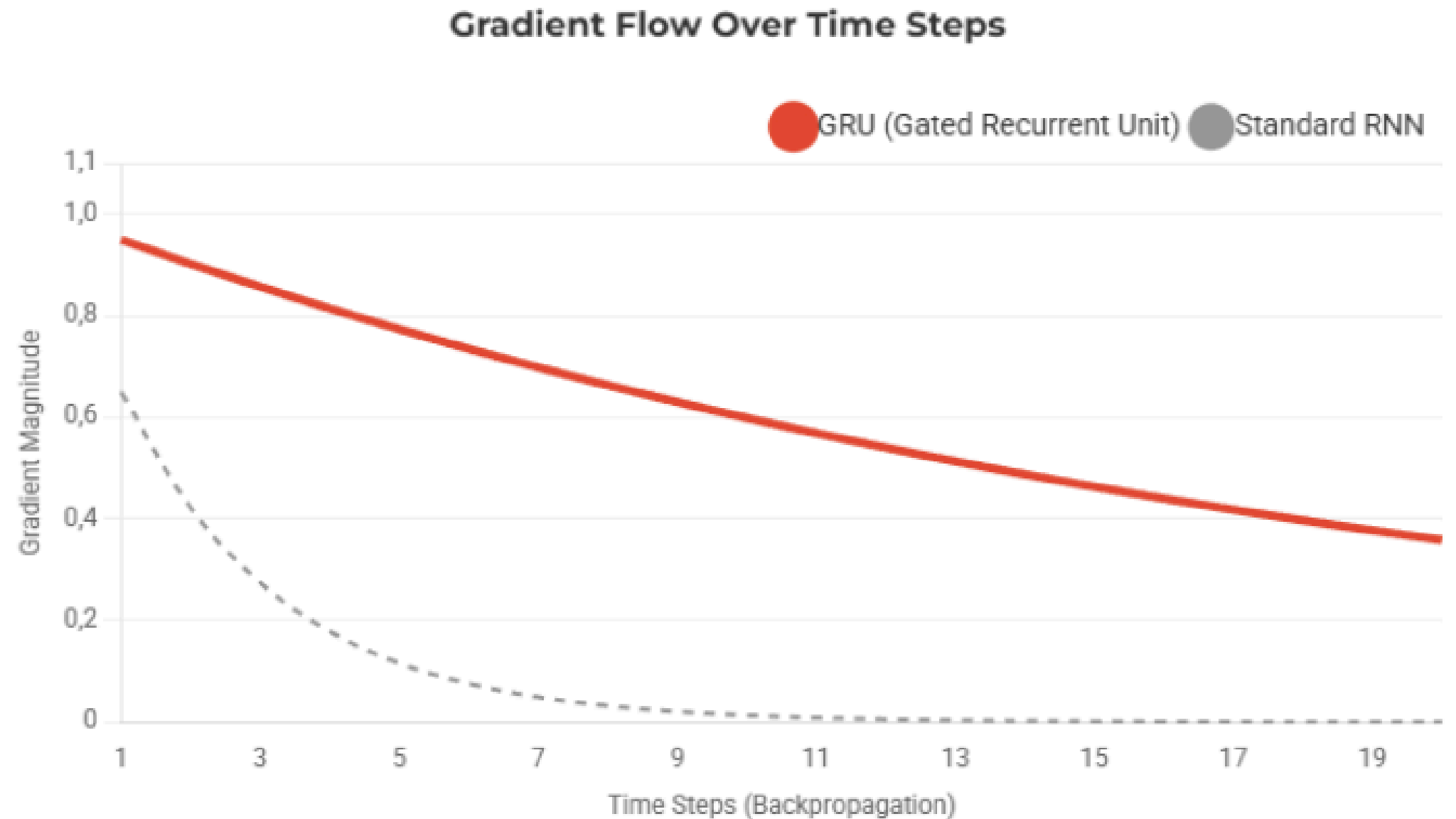
3. Հաշվողական արդյունավետություն

Շնորհիվ ավելի քիչ պարամետրերի, GRU-ն ավելի արագ է ուսուցանվում և պահանջում է ավելի քիչ հաշվողական ռեսուրսներ, պահպանելով բարձր ճշգրտություն:

Vanishing Gradient խնդիրը

Ավանդական RNN-ները դժվարանում են սովորել երկարաժամկետ հաջորդականությունների, քանի որ դրանց դեպքում, բազմակի բազմապատկումների հետևանքով, գրադիենտը դառնում է անվերջ փոքր և ձգտում զրոյի: Մոդելի կշիռները դադարում են թարմացվել, և ցանցը չի կարողանում գտնել կապը հեռու գտնվող բառերի կամ իրադարձությունների միջև (Long-term dependencies):

GRU-ն լուծում է այս խնդիրը՝ օգտագործելով **Update Gate** մեխանիզմը, որը թույլ է տալիս ցանցին որոշել, թե որքան ինֆորմացիա պահպանել նախորդ քայլերից՝ ապահովելով գրադիենտի կայուն հոսքը:



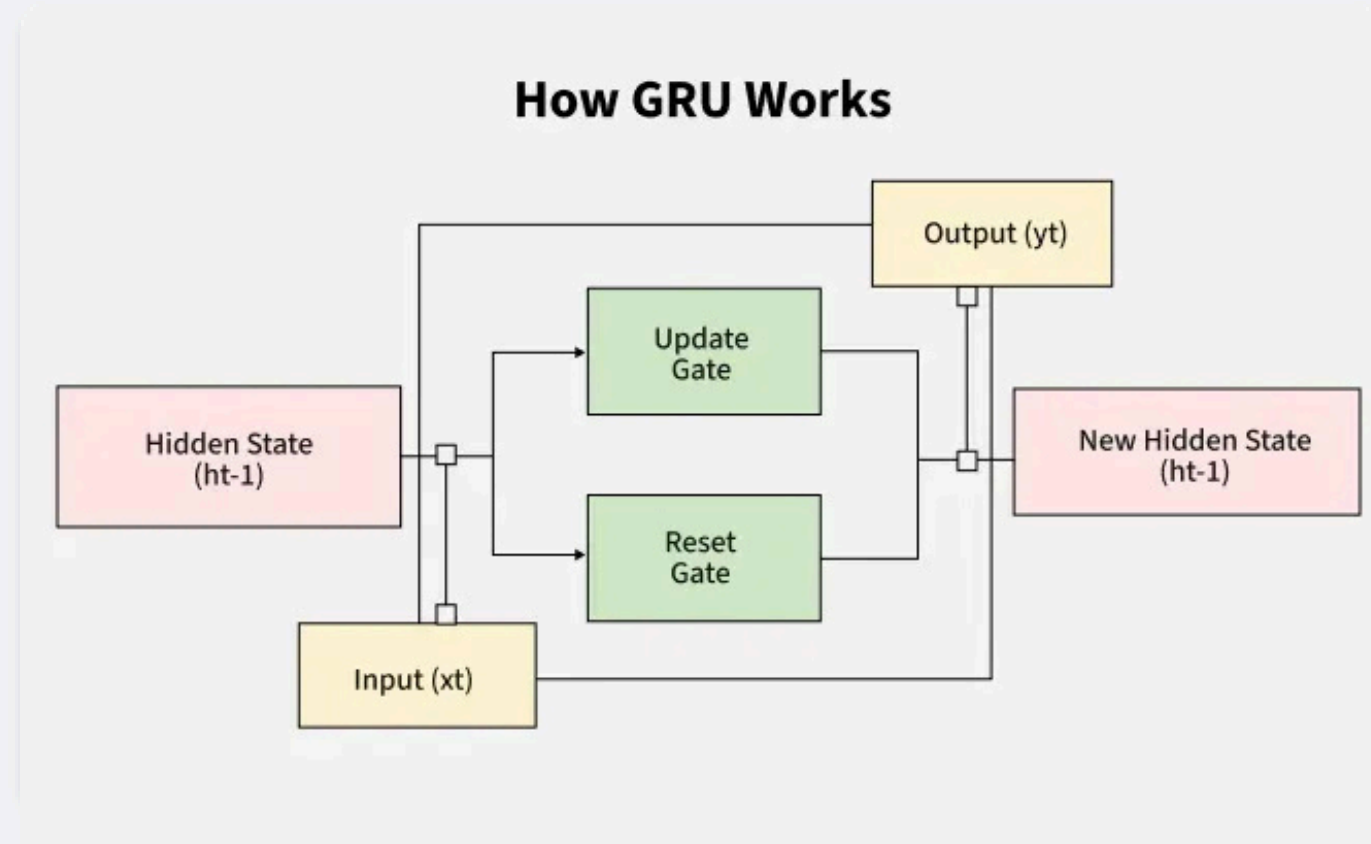
Ինչպես է աշխատում GRU-ն

Ի տարբերություն LSTM-ի, GRU-ն չունի առանձին cell state: Այն օգտագործում է մեկ միասնական hidden state, որը վերահսկվում է երկու դարպասների (gates) միջոցով:

- Update Gate (z_t): Որոշում է, թե որքան նախորդ տեղեկատվություն պետք է փոխանցվի ապագային: Այն գործում է որպես «ֆիլտր»: Եթե դարպասը փակ է (մոտ է 0-ին), նախորդ հիշողությունը պահպանվում է առանց փոփոխության:
- Reset Gate (r_t): Որոշում է, թե նախորդ տեղեկատվության որ մասը պետք է մոռացվի: Այն օգտագործվում է նոր թեկնածու վիճակը (candidate hidden state) հաշվարկելիս: Եթե մոտ է 0-ին, մոդելը «մոռանում է» նախորդ վիճակը և կենտրոնանում է միայն նոր մուտքային տվյալի (x_t) վրա: Եթե տեքստում թեման փոխվում է, Reset Gate-ը թույլ է տալիս մաքրել հին համատեքստը և սկսել նորը:

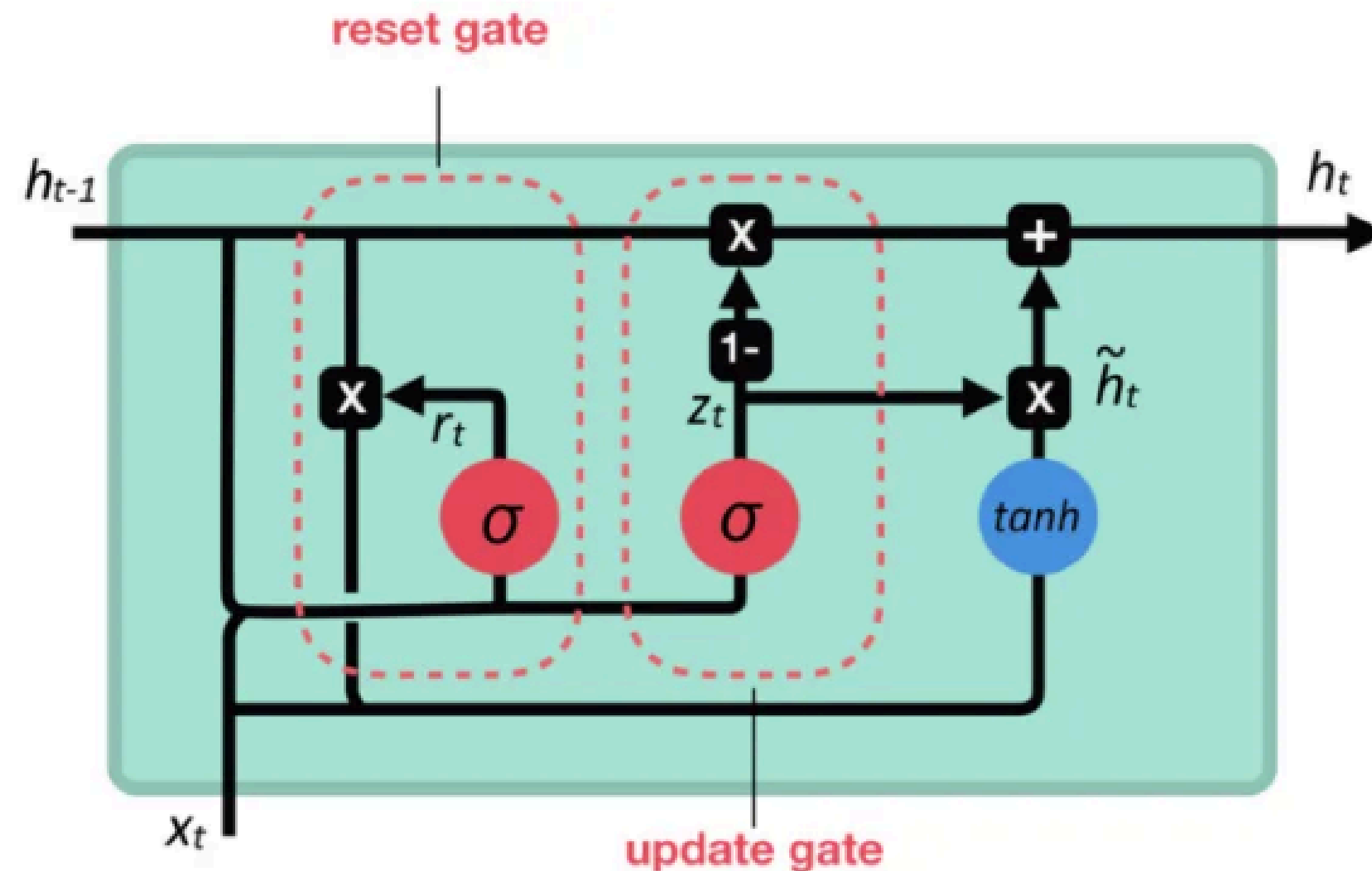
Այս մոդելում hidden state-ը միաժամանակ հանդես է գալիս որպես.

1. Short-term memory (Կարճաժամկետ հիշողություն)
2. Long-term memory (Երկարաժամկետ հիշողություն)



GRU կառուցվածք

Gated Recurrent Unit-ի ներքին կառուցվածքը և տեղեկատվության հոսքը



r_t

Reset Gate

Որոշում է, թե նախորդ hidden state-ից որքան ինֆորմացիա պետք է մոռանալ ընթացիկ քայլում:

z_t

Update Gate

Կարգավորում է հին հիշողության և նոր թեկնածուի (candidate) միջև հավասարակշռությունը:

$\tilde{h}_t$

Candidate State

Նոր ժամանակավոր հիշողություն, որը հաշվարկվում է՝ հաշվի առնելով r_t -ի ազդեցությունը:

h_t

Hidden State

Վերջնական ելքային վեկտորը, որը փոխանցվում է հաջորդ քայլին ($t+1$):

Reset Gate **Բանաձևը**

Reset Gate-ը (r_t) որոշում է, թե նախորդ թաքնված վիճակից (h_{t-1}) որքան տեղեկատվություն պետք է մոռացվի:

$$r_t = \sigma (W_r x_t + U_r h_{t-1} + b_r)$$

r_t

Reset Gate Output

Արժեքը $[0, 1]$ միջակայքում. 0 նշանակում է "մոռանալ", 1՝ "պահել"

σ

Sigmoid Function

Ակտիվացման ֆունկցիա, որը սեղմում է արժեքները 0-ի և 1-ի միջև

$W_r,$
 U_r

Weight Matrices

Reset gate-ի համար սովորվող կշիռներ (մուտքի և hidden state-ի համար)

h_{t-1}

Previous Hidden State

Նախորդ քայլի թաքնված վիճակը

Update Gate Բանաձևը

Update gate-ը (z_t) որոշում է, թե նախորդ վիճակի որ մասը պետք է փոխանցվի հաջորդ քայլին:

$$z_t = \sigma (W_z x_t + U_z h_{t-1} + b_z)$$



z_t

Update Gate Vector

Թարմացման դարպաս $[0, 1]$ միջակայքում

σ

Sigmoid Function

Ակտիվացման ֆունկցիա, սեղմում է արժեքները 0-ի և 1-ի միջև

W_z, U_z

Weight Matrices

Update gate-ի համար սովորվող կշիռներ

Interpretati
on

Իմաստը

Կշռում է նախորդ և նոր տեղեկության
հավասարակշռությունը

Candidate Hidden State

Ժամանակավոր հիշողության վիճակը, որտեղ r_t -ն վերահսկում է անցյալի ազդեցությունը:



$$\hat{h}_t = \tanh (W_h x_t + U_h (r_t \odot h_{t-1}) + b_h)$$

$\hat{h}_t$

Candidate State

Նոր հիշողության «թեկնածու» վիճակը $[-1, 1]$ միջակայքում

r_t

Reset Gate

Որոշում է՝ որքանով «մոռանալ» նախորդ վիճակը (եթե $r_t \approx 0$)

$\tanh$

Activation

Ոչ գծային ֆունկցիա, որը սեղմում է արժեքները -1 -ից 1

$\odot$

Hadamard Product

Տարր առ տարր բազմապատկում (Element-wise multiplication)²⁰

Final Hidden State **Թարմացում**

GRU-ի վերջնական քայլը՝ գծային ինտերպոլացիա (convex combination) հին հիշողության և նոր թեկնածու վիճակի միջև, որն ապահովում է կայուն gradient հոսք:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \hat{h}_t$$

"Եթե $z_t \approx 0$, մենք պահում ենք հինը: Եթե $z_t \approx 1$, մենք թարմացնում ենք նորով:"

h_t

Final Hidden State

Վերջնական թաքնված վիճակը ընթացիկ քայլում

z_t

Update Gate

Որոշում է՝ որքան տեղեկություն թարմացնել $[0, 1]$

$\hat{h}_t$

Candidate State

Նոր առաջարկվող թաքնված վիճակը (New Memory)

$\odot$

Hadamard Product

Վեկտորների տարրային բազմապատկում

GRU VS LSTM: ՀԱՄԵՄԱՏԱԿԱՆ ՎԵՐԼՈՒԾՈՒԹՅՈՒՆ



Ընտրեք GRU

Երբ առաջնային է արագությունը

- ✓ Սահմանափակ հաշվողական ռեսուրսներ (CPU, Mobile)
- ✓ Ավելի փոքր տվյալների բազաներ (Small Datasets)
- ✓ Անհրաժեշտ է արագ ուսուցում և ինֆերենս
- ✓ Պարզ հաջորդական կախվածություններ

2 vs 3

Gates Architecture

Update & Reset vs Input, Forget, Output

GRU-ն պարզեցնում է մոդելը՝ միավորելով մուտքի և մոռացման դարպասները մեկ Update Gate-ի մեջ:

No Cell

Memory State

Hidden State Only

Ի տարբերություն LSTM-ի, GRU-ն չի օգտագործում առանձին Cell State, ինչը նվազեցնում է բարդությունը:



Ընտրեք LSTM

Երբ առաջնային է հիշողությունը

- ✓ Շատ երկարաժամկետ կախվածություններ
- ✓ Մեծածավալ տվյալների հավաքածուներ (Big Data)
- ✓ Բարդ լեզվական մոդելավորում (Complex NLP)
- ✓ Երբ մոդելի ճշգրտությունը գերակա է արագությանը

Faster

Complexity & Speed

~25% Fewer Parameters

Ավելի քիչ պարամետրերը թույլ են տալիս մոդելին ավելի արագ սովորել և քիչ հիշողություն օգտագործել:

Equal

Model Accuracy

Effective for Shorter Sequences

Կարճ հաջորդականությունների դեպքում GRU-ն հաճախ գերազանցում է իսկ երկարների դեպքում՝ զիջում LSTM-ին:

GRU Կիրառություններ

01

Natural Language Processing (NLP)

GRU-ները լայնորեն կիրառվում են մեքենայական թարգմանության (Machine Translation) և տեքստի գեներացման համար:

Դրանց արդյունավետությունը թույլ է տալիս մշակել երկար նախադասություններ և պահպանել համատեքստը՝ նվազագույն ռեսուրսների օգտագործմամբ:

02

Speech Recognition

Ձայնի ճանաչման համակարգերում GRU-ները օգնում են մոդելավորել ժամանակային կախվածությունները աուդիո ազդանշաններում:

Դրանք հատկապես օգտակար են իրական ժամանակում խոսքի մշակման (Real-time Processing) և ձայնային օգնականների համար:

03

Time Series Analysis

Ժամանակային շարքերի կանխատեսում, ինչպիսիք են եղանակի տվյալները կամ ֆինանսական շուկաների միտումները:

GRU-ի ունակությունը՝ հիշել կարևոր պատմական տվյալները և անտեսել աղմուկը, այն դարձնում է իդեալական այս խնդիրների համար:

Simple Keras GRU Example (Prediction and Visualization)

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GRU, Dense

# Load data
data = pd.read_csv("sales.csv")
values = data["sales"].values.reshape(-1, 1)

# Scale
scaler = MinMaxScaler()
scaled = scaler.fit_transform(values)

# Create sequences
X, y = [], []
timesteps = 60
for i in range(timesteps, len(scaled)):
    X.append(scaled[i - timesteps:i])
    y.append(scaled[i])

X, y = np.array(X), np.array(y)

# GRU model
model = Sequential([
    GRU(50, return_sequences=True, input_shape=(timesteps, 1)),
    GRU(50),
    Dense(1)
])

model.compile(optimizer="adam", loss="mse")
model.fit(X, y, epochs=20, batch_size=32)
```

```
Epoch 1/20
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do
not pass an `input_shape` / `input_dim` argument to a layer.
5/5 ━━━━━━━━━━━ 4s 70ms/step - loss: 0.2934
Epoch 2/20
5/5 ━━━━━━━━━━━ 1s 104ms/step - loss: 0.0385
Epoch 3/20
5/5 ━━━━━━━━━━━ 1s 101ms/step - loss: 0.0374
Epoch 4/20
5/5 ━━━━━━━━━━━ 1s 106ms/step - loss: 0.0286
Epoch 5/20
5/5 ━━━━━━━━━━━ 1s 173ms/step - loss: 0.0115
Epoch 6/20
5/5 ━━━━━━━━━━━ 1s 62ms/step - loss: 0.0178
Epoch 7/20
5/5 ━━━━━━━━━━━ 0s 69ms/step - loss: 0.0154
Epoch 8/20
5/5 ━━━━━━━━━━━ 0s 64ms/step - loss: 0.0125
Epoch 9/20
5/5 ━━━━━━━━━━━ 0s 64ms/step - loss: 0.0129
Epoch 10/20
5/5 ━━━━━━━━━━━ 0s 68ms/step - loss: 0.0136
Epoch 11/20
5/5 ━━━━━━━━━━━ 0s 63ms/step - loss: 0.0102
Epoch 12/20
5/5 ━━━━━━━━━━━ 0s 62ms/step - loss: 0.0120
Epoch 13/20
5/5 ━━━━━━━━━━━ 0s 67ms/step - loss: 0.0112
Epoch 14/20
5/5 ━━━━━━━━━━━ 0s 64ms/step - loss: 0.0091
Epoch 15/20
5/5 ━━━━━━━━━━━ 0s 65ms/step - loss: 0.0114
Epoch 16/20
5/5 ━━━━━━━━━━━ 0s 68ms/step - loss: 0.0105
Epoch 17/20
5/5 ━━━━━━━━━━━ 1s 61ms/step - loss: 0.0114
Epoch 18/20
5/5 ━━━━━━━━━━━ 0s 78ms/step - loss: 0.0099
Epoch 19/20
5/5 ━━━━━━━━━━━ 0s 65ms/step - loss: 0.0097
Epoch 20/20
5/5 ━━━━━━━━━━━ 0s 62ms/step - loss: 0.0099

<keras.src.callbacks.history.History at 0x7931ee0e3470>
```

```
pred_scaled = model.predict(X)
pred = scaler.inverse_transform(pred_scaled)
actual = scaler.inverse_transform(y)
```

33/33 ————— 1s 23ms/step

Actual: 134.22 | Predicted: 124.09

Actual: 131.51 | Predicted: 124.96

Actual: 115.82 | Predicted: 125.80

Actual: 110.03 | Predicted: 126.19

Actual: 128.36 | Predicted: 126.02

```
import matplotlib.pyplot as plt
```

```
# Plot actual vs. predicted values
```

```
plt.figure(figsize=(15, 6))
```

```
plt.plot(actual, label='Actual Sales')
```

```
plt.plot(pred, label='Predicted Sales')
```

```
plt.title('Actual vs. Predicted Sales')
```

```
plt.xlabel('Time Step')
```

```
plt.ylabel('Sales')
```

```
plt.legend()
```

```
plt.show()
```

